

AI WEATHERWISE

Phase 3: Project Design

Architecture, data model, API and security design

Project	AI WeatherWise – Deliver intelligent weather forecasts and insights
Phase	3 of 8 – Project Design
Technology	Node.js, Express.js, MongoDB (Mongoose), JWT, Google Gemini AI, OpenWeatherMap
Version / Date	1.0 28 September 2026

1. Design Overview

The AI WeatherWise application follows the **MVC pattern** to decouple business logic from data management. The Controller layer orchestrates weather fetching from OpenWeatherMap and generation of AI insights via Gemini, while the Model layer manages the User and Location entities. Because this is a headless backend, the View layer is the API routing layer.

2. System Architecture

A client such as Postman sends JSON requests to the Express server gateway. Authentication middleware verifies the JWT for private routes, requests are routed to controllers, controllers call the weather and AI services, and Mongoose models persist data in MongoDB.

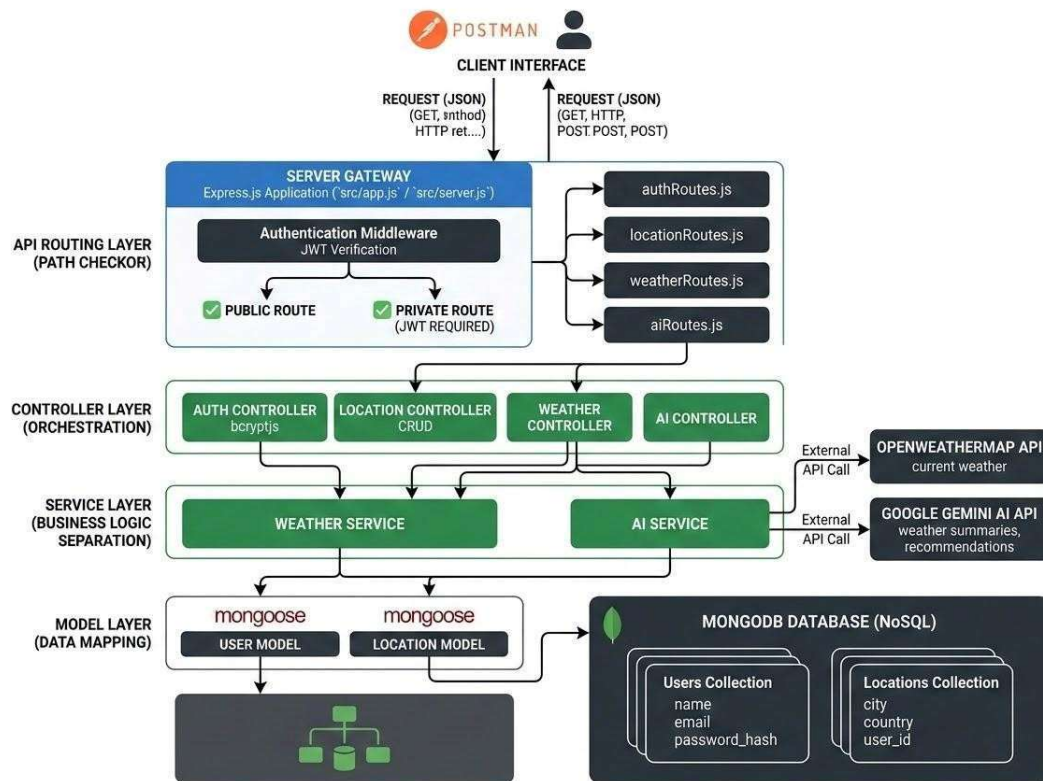


Figure 1: AI WeatherWise layered project architecture

Layer	Components	Responsibility
API routing layer	authRoutes, locationRoutes, weatherRoutes, aiRoutes; JWT authentication middleware	Maps paths to controllers; separates public and private routes.
Controller layer	Auth, Location (CRUD), Weather and AI controllers	Orchestrates requests, validates input, formats JSON output.
Service layer	Weather service, AI service	Business logic and external API calls (OpenWeatherMap, Google Gemini).
Model layer	User model, Location model (Mongoose)	Schema, validation and data mapping to MongoDB.
Database	MongoDB – Users and Locations collections	Persistent storage.

3. MVC Architecture Pattern

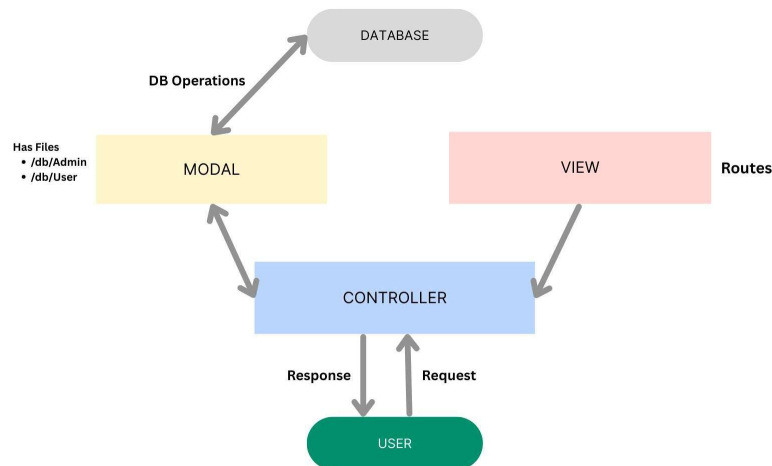


Figure 2: MVC architecture used by the project

Layer	Role in AI WeatherWise
Model (Data blueprint)	Defines strongly enforced Mongoose schemas mapped to MongoDB collections, with default states, indexing rules and validation.
Controller (Execution matrix)	Captures requests from routes, checks constraints, delegates work to models and services and packages results into standard JSON.
View (Routing interface)	No HTML templates exist; the view is the API routing layer linking network paths to controller handlers.

4. Component Definitions

Component	Description
Interface	API tools such as Postman query route paths and receive JSON entities.
Express server gateway	Listens on the network port, maps HTTP methods (GET, POST, PUT, DELETE), parses JSON payloads and handles CORS headers.
Authentication & role middleware	Extracts the token from the request headers, validates it and checks whether the role is allowed for guarded routes.
Granular services	Isolated modules for weather retrieval and AI generation.
Database interface layer	Mongoose translates application logic into native MongoDB queries.
AI service integration	Uses the Google Gemini SDK to generate weather summaries and recommendations from live temperature and conditions.

5. Data Design

5.1 Entity-Relationship Diagram

The extended ER diagram shows the complete data architecture, including entities reserved for future releases. The current release implements the **User** and **Location** entities; the remaining entities (weather data, AI insights, alerts, subscriptions, API usage logs and AI models) are planned for later versions.

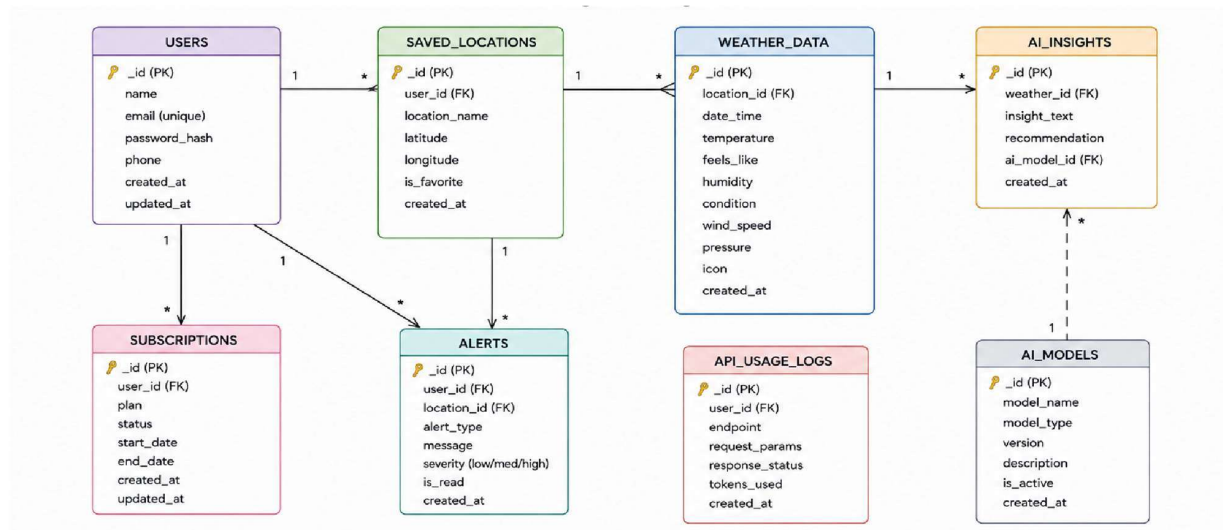


Figure 3: Entity-Relationship diagram of AI WeatherWise

5.2 Implemented Entities

Entity	Attributes	Notes
User	_id (PK), name, email (unique), password (hashed), role (default "reader"), createdAt, updatedAt	Owns many locations.
Location	_id (PK), user (FK to User), city, country, createdAt	Unique index on user + city + country.

Relationship: User to Location is **one-to-many** – a registered user can save multiple favorite locations.

6. API Design

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Register a user; returns id, name, email and JWT (201 Created).
POST	/api/auth/login	Public	Authenticate a user; returns user data and JWT (200 OK).
GET	/api/weather/:city	Public	Current weather: city, temperature, humidity, windSpeed, condition, isMock.
POST	/api/ai/weather-recommendation	JWT (per user flow)	Takes city, temperature, humidity and condition; returns an AI recommendation.
POST / GET	/api/locations	JWT	Add a favorite city / list the user's favorite cities (proposed route).
PUT / DELETE	/api/locations/:id	JWT	Update or remove a favorite city (proposed route).

Standard response format: every response is JSON with a `success` flag, followed by `data` (or `recommendation`) on success and a `message` on failure.

7. User Flow

1. User logs in or registers.
2. User saves a favorite city via the API.
3. User fetches current weather metrics for that city.
4. User requests AI-driven insights (summaries / recommendations) based on the weather data.
5. System validates the JWT and returns the requested information.

AI WeatherWise API User Workflow

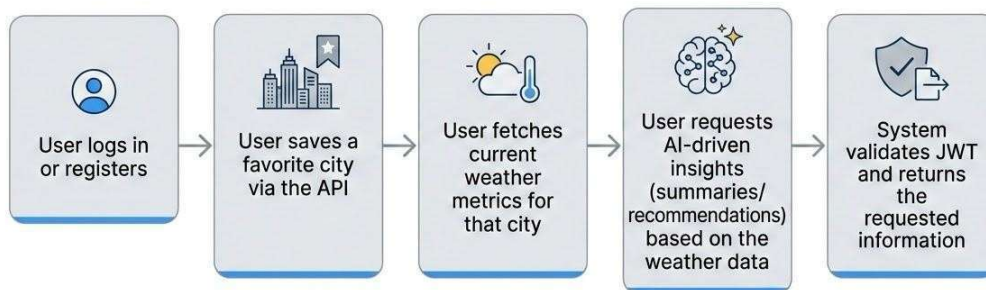


Figure 4: AI WeatherWise API user workflow

8. Security Design

- **Authentication:** JWT issued at login/registration and verified by middleware on private routes.
- **Password protection:** bcryptjs hashing; minimum password length of six characters.
- **Authorization:** role-based route guarding blocks unauthorized tiers from modifying core settings.
- **Input safety:** request sanitization prevents XSS and NoSQL injection.
- **Secrets:** JWT secret and API keys are stored in `.env`, which is excluded from Git through `.gitignore`.

9. Resilience and Error Handling Design

- **Fallback mode:** if external API keys are absent, weather and AI services return mock data and the weather response carries `isMock: true`.

- **Fail-fast start-up:** the server starts only after the MongoDB connection succeeds; otherwise the process exits with an error message.
- **Centralized error handler:** returns uniform JSON error responses.

10. Project Structure Design

The source code is organized by responsibility so that each concern can be maintained independently.

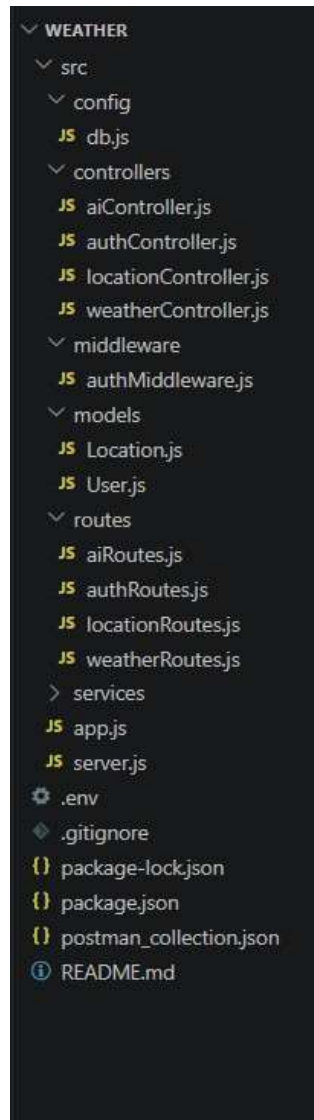


Figure 5: Backend folder structure

Folder / file	Content
src/config	db.js – MongoDB connection
src/controllers	authController, locationController, weatherController, aiController
src/middleware	authMiddleware – JWT verification
src/models	User.js, Location.js
src/routes	authRoutes, locationRoutes, weatherRoutes, aiRoutes
src/services	Weather and AI service modules

Folder / file	Content
src/app.js, src/server.js	Express app configuration and server start-up
Root files	.env, .gitignore, package.json, postman_collection.json, README.md

The next phase, **Project Planning**, schedules the work needed to build this design.